

THREE ROADS: PORTFOLIO STACK DECISION

1. PORTFOLIO CONSTRAINTS

Before choosing a technology stack, I gave AI my actual constraints instead of asking it to simply choose a technology for me.

1. Free Only

I need a solution that can be built and hosted completely for free. I do not want to depend on paid hosting, paid databases, or paid software while building my portfolio.

2. My Honest Skill Level

I am a final-year Computer Systems Engineering student. My strongest areas are Python, C/C++, HTML, CSS, JavaScript, Git/GitHub, and introductory-to-intermediate AI/ML development. I have also worked with technologies such as Laravel, MySQL, and different AI/ML frameworks.

I am comfortable writing and modifying code, but I do not want to choose a complicated stack just because it looks more advanced.

3. What My Portfolio Needs to Do

My portfolio needs to present me as a Computer Systems Engineering student interested in Artificial Intelligence, Machine Learning, software development, and research. The main pages/content I need are:

- Home / Landing page
- About Me
- Projects
- Research / Experience
- Case studies
- Skills
- Contact
- Links to GitHub, LinkedIn, and other professional profiles

The portfolio should guide a visitor from discovering who I am to understanding my work and eventually contacting me or viewing my projects.

4. How My Work Must Be Displayed

My projects need to be displayed using:

- Project images and screenshots
- Image galleries where useful
- Embedded demos when available
- Links to GitHub repositories
- Technical project descriptions
- Long-form case studies for important projects
- Research/project documentation

At this stage, I do not need a backend or database. The portfolio mainly needs to present information and link visitors to my projects and external resources.

2. THREE STACK OPTIONS

Option 1 – HTML + CSS + JavaScript + GitHub Pages

HOW I WOULD BUILD IT

I would create the portfolio using standard HTML, CSS, and JavaScript. The website would be stored in a GitHub repository and deployed using GitHub Pages.

FREE HOSTING

GitHub Pages

BACKEND REQUIRED?

No. The website can be completely static.

ADVANTAGES

- Very simple architecture
- Completely free
- Easy to understand and modify
- Fast loading
- Easy to connect with GitHub repositories
- No database or server maintenance
- Good for images, project pages, case studies, and links

TRADE-OFF

I would have to manually update the website when I add projects or change content. More advanced features would require me to write additional JavaScript or introduce another service.

Option 2 – Next.js + GitHub + Vercel

HOW I WOULD BUILD IT

I would build the portfolio using Next.js and React. I could create reusable components for projects, case studies, navigation, and other sections. The project would be stored on GitHub and deployed through Vercel's free tier.

FREE HOSTING

Vercel free tier

BACKEND REQUIRED?

Not initially. Next.js can support backend functionality later, but I do not currently need it.

ADVANTAGES

- More scalable than a basic static website
- Component-based development
- Good developer experience
- Easy GitHub integration
- Can add dynamic functionality later
- Suitable for a modern professional portfolio

TRADE-OFF

It introduces more complexity than I currently need. I would need to maintain React/Next.js dependencies and understand the framework properly. For a simple portfolio, some of this complexity would be unnecessary.

Option 3 – Next.js + Backend/API + Database

HOW I WOULD BUILD IT

I could create a full-stack application using Next.js with backend/API functionality and a database such as Supabase. This could support dynamic projects, contact forms, authentication, analytics, or an admin dashboard.

FREE HOSTING

I could initially use Vercel + Supabase free tiers, subject to their current limits.

BACKEND REQUIRED?

Yes. This would introduce a backend/API and database.

ADVANTAGES

- Most powerful option
- Dynamic content
- Database support
- Could add an admin dashboard
- Could add authentication
- Could support a dynamic blog
- Could later integrate AI-powered features

TRADE-OFF

This is much more complicated to build and maintain. It introduces database management, backend logic, security considerations, deployment configuration, and additional dependencies. For my current portfolio requirements, this would be over-engineering.

3. PRESSURE TEST

What breaks if I choose the simplest option?

If I choose HTML, CSS, JavaScript, and GitHub Pages, I will not have a built-in database or backend.

If I eventually want an admin panel where I can log in and add projects without editing code, I would need to introduce additional tools or migrate to a more dynamic architecture. However, this does not prevent me from achieving my current portfolio goals.

My projects can still have screenshots, galleries, GitHub links, embedded demos, and detailed case-study pages. Therefore, the limitations of the simplest option are acceptable for my current needs.

What would I maintain if I chose the most powerful option?

With the full-stack option, I would need to maintain:

- Frontend code
- Backend/API code
- Database
- Environment variables
- Authentication/security if added

- Dependencies
- Deployment configuration
- Database structure and backups

That is significantly more maintenance for a portfolio that currently does not require those features.

Can I finish it in two weeks?

Yes, with the simplest option. HTML, CSS, JavaScript, and GitHub Pages allow me to focus on the actual content and presentation rather than spending most of the two weeks learning and configuring a complex framework.

The Next.js option is also possible within two weeks, but it gives me less margin for content creation and polishing.

The full-stack option would make completing the portfolio within two weeks more risky because of the additional development and debugging required.

Does it show my work well?

Yes. The simplest stack can display project screenshots, image galleries, GitHub repositories, embedded demos, research work, long-form case studies, technical explanations, and professional links.

These are the most important requirements for my portfolio, so I do not need a backend just to present them effectively.

MY FINAL DECISION

CHOSEN STACK: HTML + CSS + JAVASCRIPT + GITHUB PAGES

I chose HTML, CSS, JavaScript, GitHub, and GitHub Pages for my portfolio.

I did not choose Next.js because, although it is more powerful and modern, I do not currently need its additional complexity. My portfolio is primarily a content and project showcase, so a static website is enough for what I need.

I also did not choose the full-stack Next.js + backend + database option because it would be unnecessary at this stage. I do not need user accounts, an admin dashboard, a database, or dynamic project management yet.

The most important question for me is “*can I maintain this?*” With HTML, CSS, and JavaScript, the answer is yes. I already understand these technologies, so I can update my projects, case studies, images, and links without depending on a complicated backend.

The second important question is “*does it show my work well?*” The answer is also yes. I can create attractive project pages, image galleries, embedded demonstrations, GitHub links, and detailed case studies.

I am choosing the simplest technology not because it is the most powerful, but because it is powerful enough for my actual needs, free, maintainable, and realistic to complete within two weeks.

If my requirements become more dynamic in the future, I can migrate to Next.js or add backend functionality later.